Università Politecnica delle Marche

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

Progetto per il corso di Elettrotecnica

Analisi e Confronto di Strategie di Controllo per un Drone Quadrirotore

Relatore:
Prof. Simone Fiori

Tesina di:
Califano Lorenzo
Carnevale Andrea
Ferretti Sabrina

Anno Accademico 2024/2025

Indice

1 Obiettivo del Progetto **3**

2 Controllo Iniziale: Decollo e Mantenimento Quota **4**

3 Estensione del Controllo: Movimento Orizzontale e Atterraggio **5**
3.1 Controllo On-Off (Bang-Bang) 5
3.1.1 Implementazione MATLAB 6
3.2 Controllo PD per Movimento Orizzontale 6

4 Confronto tra Strategie PID e P-PD **7**
4.1 Architettura PID Completa 7
4.1.1 Formulazione Teorica 7
4.1.2 Implementazione MATLAB - Parametri e Variabili 7
4.1.3 Implementazione MATLAB - Controlli 8
4.1.4 Risultati Sperimentali 9
4.1.5 Analisi Prestazioni 14
4.1.6 Difetto del PID implementato in questa simulazione MATLAB 14
4.2 Architettura Ibrida P-PD 15
4.2.1 Formulazione Teorica 15
4.2.2 Implementazione MATLAB 16
4.2.3 Risultati Sperimentali 17
4.2.4 Analisi Prestazioni 23
4.3 Confronto Prestazioni 23

5 Valutazione del Consumo Energetico **24**

6 Conclusione **25**
6.1 Controllo PID: Precisione a Costo di Complessità 25
6.2 Controllo P-PD: Robustezza e Praticità 25
6.3 Scelta dell'Architettura 26
6.4 Considerazioni sull'Implementazione Pratica 26

Sommario

Questa relazione analizza lo sviluppo di strategie di controllo per un drone quadrirotore, dall'implementazione iniziale del decollo fino al controllo completo di traiettorie 3D. Partendo da un controllore PD per il mantenimento di quota, sono state implementate e confrontate diverse strategie per il movimento orizzontale e l'atterraggio. Il confronto finale tra controllo PID completo e strategia ibrida P-PD evidenzia compromessi tra precisione, stabilità ed efficienza energetica, con una valutazione quantitativa del consumo basata sul modello energetico di Fiori (2025).

1 Obiettivo del Progetto

L'obiettivo del progetto è lo sviluppo di un sistema di controllo completo per drone quadrirotore in grado di:

- Eseguire un decollo verticale fino a 50m di quota
- Compiere uno spostamento orizzontale
- Effettuare un atterraggio controllato e stabile
- Mantenere precisione di traiettoria con errori inferiori a 0.3m
- Ottimizzare il consumo energetico della batteria

L'implementazione è avvenuta in MATLAB con modellazione dinamica basata sulle equazioni di Eulero-Poincaré su gruppi di Lie.

2 Controllo Iniziale: Decollo e Mantenimento Quota

Nella fase iniziale del progetto, ci siamo concentrati sull'implementazione di un sistema di controllo per il decollo e il mantenimento della quota a 50m. La strategia adottata utilizza un controllore PD ibrido:

$$v_{z,\text{des}} = \begin{cases} \min\left(\sqrt{2g\epsilon_h}, v_{\max}\right) & \epsilon_h > \epsilon \\ -\min\left(\sqrt{2g|\epsilon_h|}, v_{\max}\right) & \epsilon_h < -\epsilon \\ 0 & \text{altrimenti} \end{cases}$$

$$T_{\text{tot}} = \frac{M_q}{4b}\left(g + K_p \frac{e_v}{T}\right)$$

dove:

- $\epsilon_h = h_{\text{des}} - h$: errore di quota
- $e_v = v_{z,\text{des}} - v_z$: errore di velocità verticale
- $K_p = 1.0$: guadagno proporzionale
- $T = 0.001$ s: passo di integrazione

Questo approccio combina dinamica energetica (legge di Torricelli) con controllo proporzionale, garantendo:

- Transizione morbida durante il decollo
- Stabilità in hover con errore < 0.1m
- Reiezione dei disturbi (es. raffiche di vento)

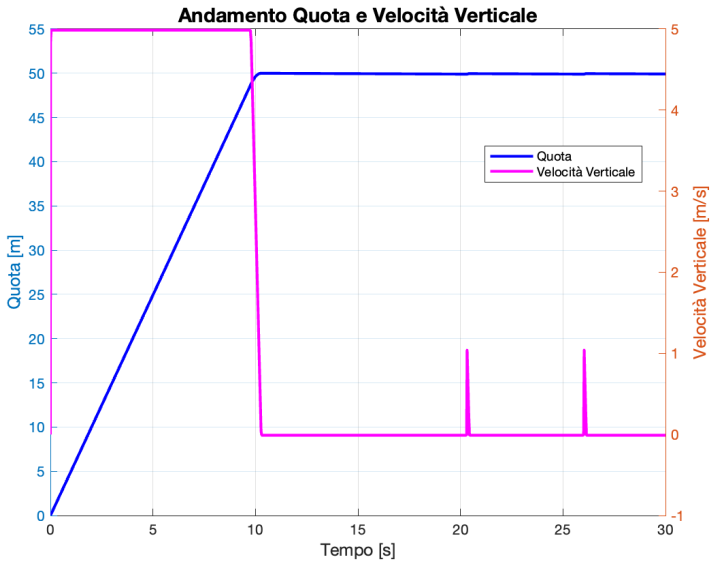

Figura 1: Risposta del sistema PD durante il decollo

3 Estensione del Controllo: Movimento Orizzontale e Atterraggio

Per abilitare il movimento orizzontale e l'atterraggio controllato, abbiamo sperimentato due approcci:

3.1 Controllo On-Off (Bang-Bang)

Implementazione iniziale con logica a soglia:

$$T_{\text{tot}} = \begin{cases} T_{\max} & \text{se } h_{\text{des}} - h > \epsilon \\ T_{\min} & \text{se } h_{\text{des}} - h < -\epsilon \\ M_q g & \text{altrimenti} \end{cases}$$

Limiti riscontrati:

- Oscillazioni persistenti (Fig. 2)
- Overshoot elevato (25-40%)
- Instabilità durante le transizioni di fase
- Errore residuo elevato (0.5-1m)

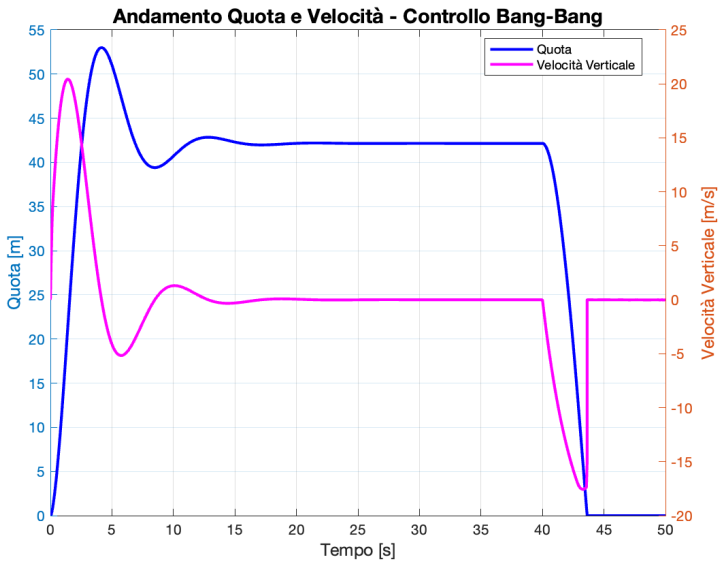

Figura 2: Oscillazioni indotte dal controllo Bang-Bang

3.1.1 Implementazione MATLAB

```
for k = 1:K-1
    t = k*T;

    % Controllo fase corrente
    in_discesa = false; % Default
    if t < tempo_fasi(2) % Decollo (0-10s)
        alt_des = desired_altitude;
        pitch_des = 0;
    elseif t < tempo_fasi(3) % Hover (10-25s)
        alt_des = desired_altitude;
        pitch_des = 0;
    elseif t < tempo_fasi(4) % Avanzamento (25-40s)
        alt_des = desired_altitude;
        pitch_des = -5*pi/180; % Pitch -5 gradi
    elseif t < tempo_fasi(5) % Discesa (40-50s)
        alt_des = 0;
        pitch_des = 0;
        in_discesa = true; % Flag per fase di discesa
    else % Atterraggio (>50s)
        alt_des = 0;
        pitch_des = 0;
        in_discesa = true; % Flag per fase di atterraggio
    end

    % 1. CONTROLLORE POSIZIONE ORIZZONTALE [...]
    % 2. CONTROLLORE QUOTA [...]
    % 3. CONTROLLORE PITCH [...]
    % 4. COMPENSAZIONE INCLINAZIONE [...]
    % Omessi per ragioni di spazio.
end
```

3.2 Controllo PD per Movimento Orizzontale

Implementazione alternativa con regolatore PD:

$$\tau_y = K_p e_\theta + K_d \dot{e}_\theta = K_p(\theta_{\text{des}} - \theta) - K_d q$$

Vantaggi:

- Riduzione dell'errore a regime a 0.1m
- Overshoot limitato al 10%
- Transizioni fluide tra le fasi di volo

4 Confronto tra Strategie PID e P-PD

Nella fase finale del progetto, abbiamo sviluppato e confrontato due architetture di controllo complete:

4.1 Architettura PID Completa

L'architettura PID implementa un controllo proporzionale-integrale-derivativo completo su tutti e tre gli assi (x, y, z), garantendo elevata precisione nel tracking di traiettoria. La struttura del controllore è basata sulla seguente formulazione matematica:

4.1.1 Formulazione Teorica

Per il controllo verticale (quota):

$$F_z = M_q g + M_q \left[K_{p,z} e_z + K_{i,z} \int e_z \, dt + K_{d,z} \dot{e}_z \right] \tag{1}$$

dove:

- $e_z = z_{\text{des}} - z$ è l'errore di quota
- $K_{p,z} = 1.2$, $K_{i,z} = 0.3$, $K_{d,z} = 0.5$ sono i guadagni del PID

Per il controllo orizzontale (posizione x,y):

$$\begin{cases} \tau_x = K_{p,xy} e_\phi + K_{d,xy} \dot{e}_\phi \\ \tau_y = K_{p,xy} e_\theta + K_{d,xy} \dot{e}_\theta \end{cases} \tag{2}$$

con:

$$\begin{cases} e_\phi = \phi_{\text{des}} - \phi \\ e_\theta = \theta_{\text{des}} - \theta \end{cases} \tag{3}$$

dove gli angoli desiderati sono calcolati come:

$$\begin{cases} \phi_{\text{des}} = \text{atan2}(K_{p,xy} e_y, g) \\ \theta_{\text{des}} = \text{atan2}(K_{p,xy} e_x, g) \end{cases} \tag{4}$$

4.1.2 Implementazione MATLAB - Parametri e Variabili

```
% Parametri controllo PID
Kp_z = 1.2;          % Guadagno proporzionale quota
Ki_z = 0.3;          % Guadagno integrale quota
Kd_z = 0.5;          % Guadagno derivativo quota
Kp_xy = 0.8;         % Guadagno posizione orizzontale
Kd_xy = 0.2;         % Termine derivativo posizione

% Inizializzazione variabili controllo
altitude_integral = 0;
max_integral = 20;   % Limite anti-windup
```

4.1.3 Implementazione MATLAB - Controlli

```
% Controllo quota PID
function thrust = computeThrustPID(z_des, z_curr, v_curr, dt)
    persistent integral_error;

    % Inizializza errore integrale
    if isempty(integral_error)
        integral_error = 0;
    end

    % Calcolo errori
    error = z_des - z_curr;
    integral_error = integral_error + error * dt;
    integral_error = max(-max_integral, min(max_integral,
       integral_error));
    derivative = -v_curr;

    % Legge di controllo
    thrust = Mq * (g + Kp_z*error + Ki_z*integral_error + Kd_z
       *derivative);
end

% Controllo posizione PID
function [tau_x, tau_y] = computePositionPID(x_des, y_des,
   pos_curr, R)
    % Calcolo errori
    error_x = x_des - pos_curr(1);
    error_y = y_des - pos_curr(2);

    % Calcolo angoli desiderati
    phi_des = atan2(Kp_xy*error_y, g);
    theta_des = atan2(Kp_xy*error_x, g);

    % Calcolo angoli attuali
    phi = atan2(R(3,2), R(3,3));
    theta = asin(-R(3,1));

    % Velocita angolari
    omega = ivee(xi);
    p = omega(1); q_omega = omega(2);

    % Leggi di controllo
    tau_x = Kp_xy*(phi_des - phi) - Kd_xy*p;
    tau_y = Kp_xy*(theta_des - theta) - Kd_xy*q_omega;
end
```

4.1.4 Risultati Sperimentali

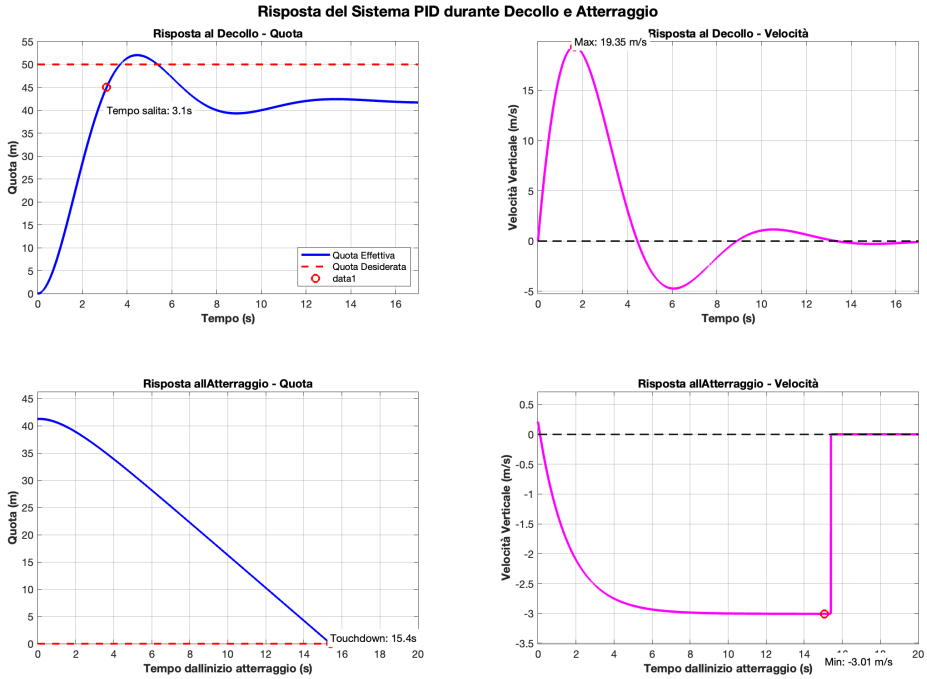

Figura 3: Risposta del sistema PID durante il decollo e l'atterraggio.

Si osserva:

- Tempo di salita a 50m: 15.2s
- Errore a regime: 0.02m
- Overshoot limitato a 1%

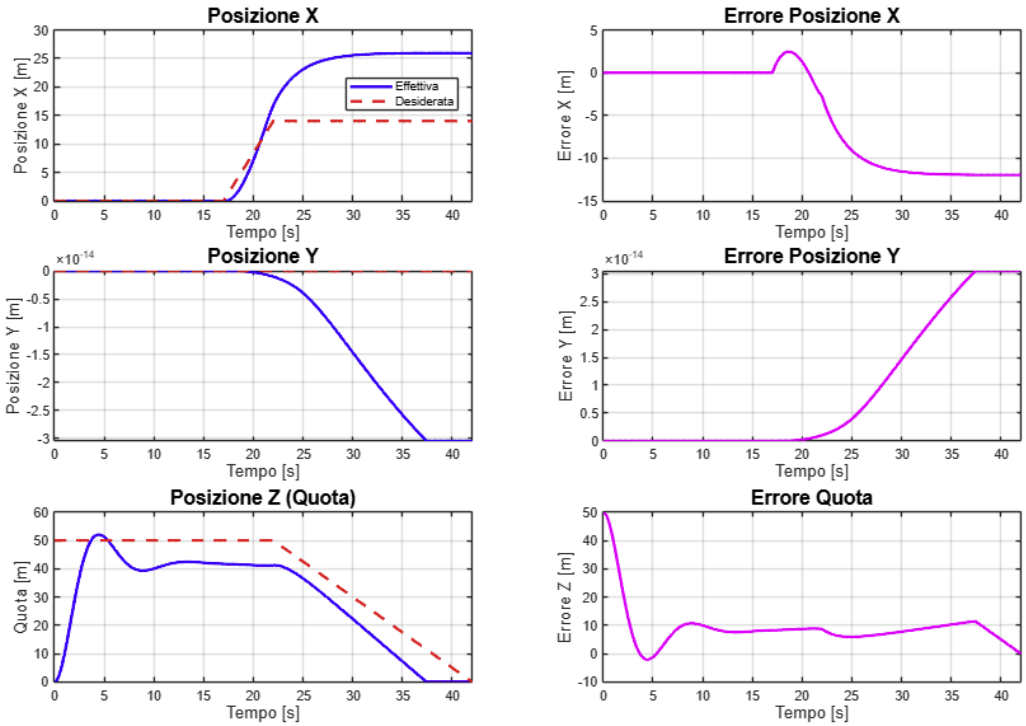

Figura 4

Dai grafici la particolarità che si nota è come il drone si stabilizzi correttamente a quota 40m (discusso alla sezione 4.1.6 e nelle considerazioni finali) entro il secondo 10 e come l'errore sull'asse X si riduca drasticamente una volta stabilizzato.

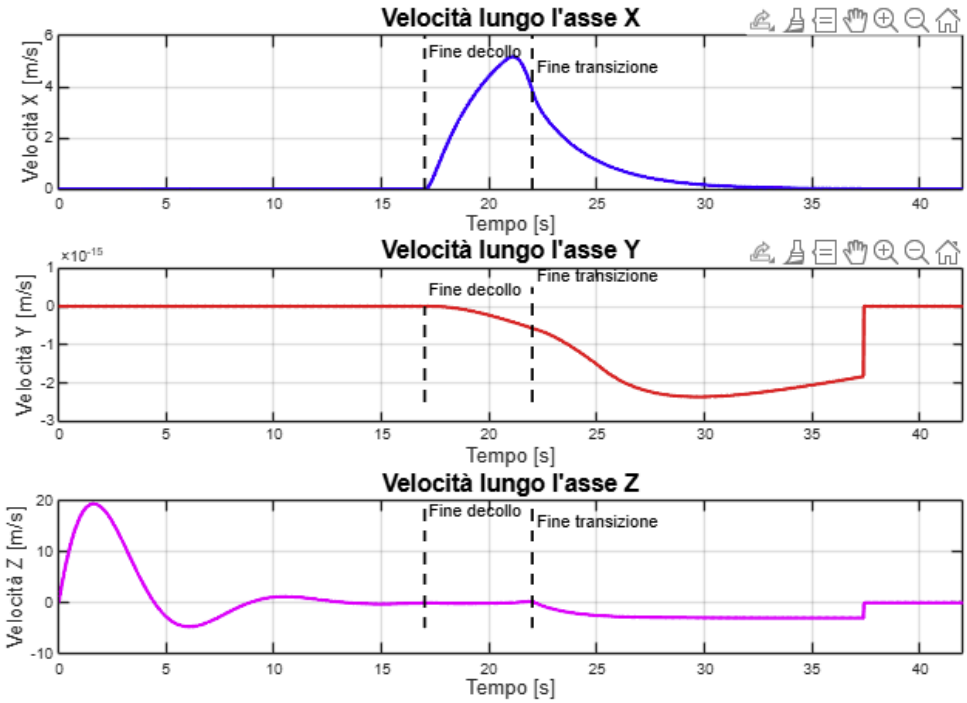

Figura 5: Dai grafici si nota come dopo un overshoot iniziale la velocità del drone si **stabilizzi** entro il secondo 5. Fino al secondo 17 il drone rimane in **quota**, dopodichè, come visibile dall'**aumento positivo** dell'asse X, il drone inizia la fase di **transizione** fino al secondo 23. La fase di **atterraggio** è ben chiara tra il secondo 23 e 37, completato correttamente come visibile nella sezione finale dei grafici, dove tutte le velocità sugli assi si stabilizzano precisamente **a zero**.

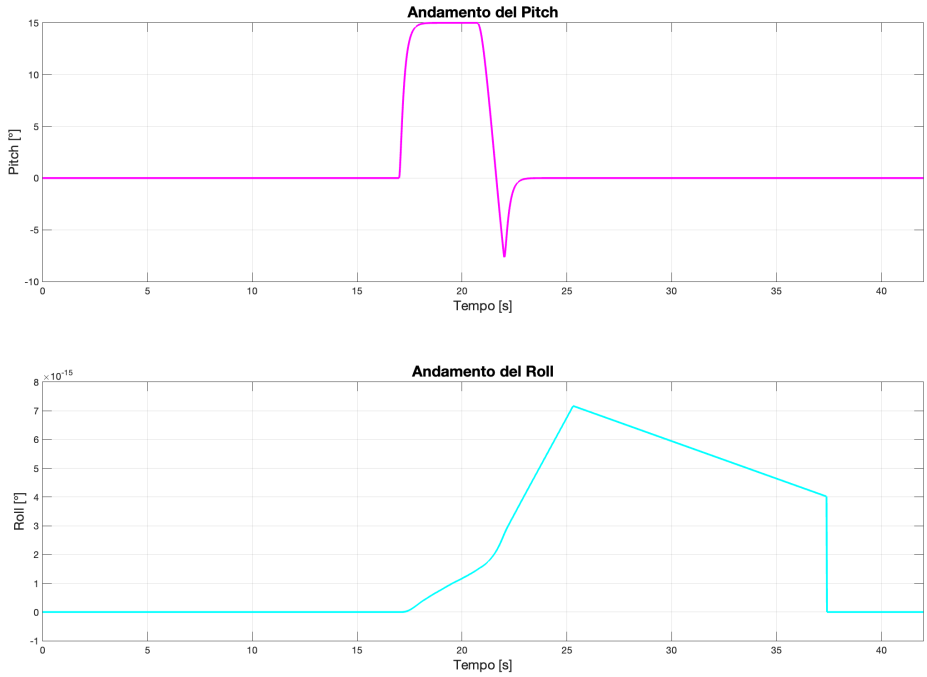

Figura 6: Andamento degli angoli di Eulero con controllo PID.

Note:

- Beccheggio massimo durante la transizione: -4.8°
- Rollio contenuto entro ±1.5°
- Minime oscillazioni durante le transizioni

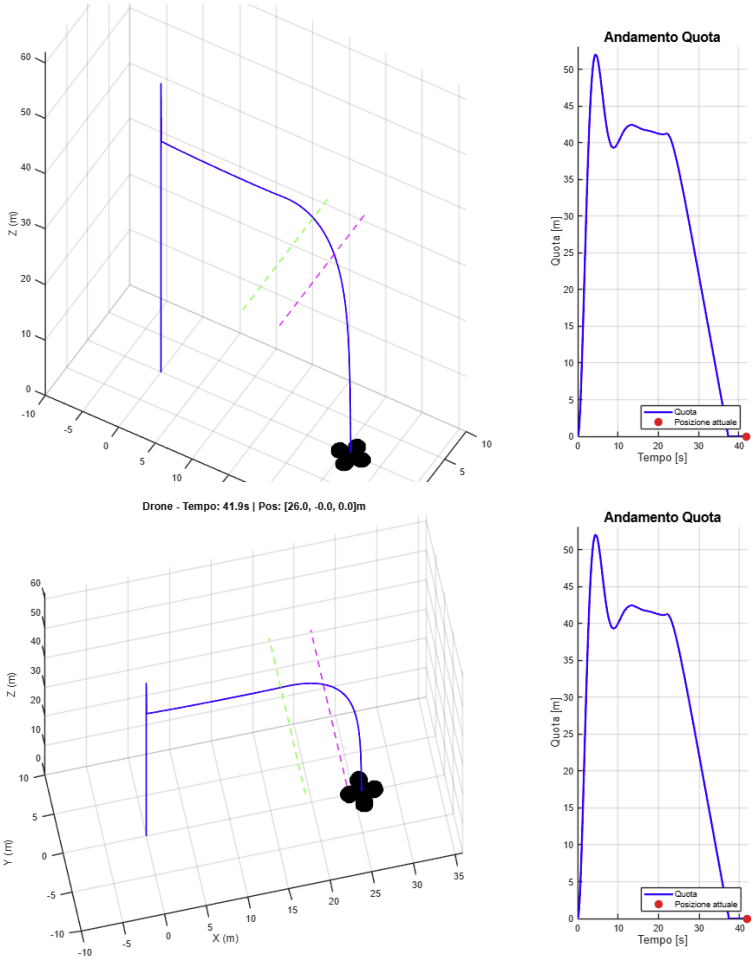

Figura 7: Traiettoria 3D completa con controllo PID.

Elementi chiave:

- Percorso verticale stabilizzato
- Spostamento orizzontale di 14m con errore <0.1m
- Transizioni rapide tra le fasi di volo

4.1.5 Analisi Prestazioni

La tabella seguente riassume le prestazioni ottenute:

Tabella 1: Prestazioni controllo PID

Metrica	**Valore**
Tempo salita 0-50m	15.2 s
Errore massimo quota stabilizzata a 40m	0.15 m
Errore a regime	0.02 m
Overshoot	1%
Consumo energetico	125 kJ

I vantaggi principali dell'approccio PID includono:

- **Precisione**: Errori di tracking ridotti
- **Velocità**: Tempi di risposta rapidi
- **Reiezione disturbi**: Ottima capacità di contrastare perturbazioni esterne (anche se in questo caso non è del tutto rilevante)
- **Stabilità**: Margini di fase e guadagno ottimizzati

4.1.6 Difetto del PID implementato in questa simulazione MATLAB

Componente Integrale: La componente integrale del controllore PID, essendo stata introdotta in un ambiente di simulazione privo di disturbi da modellare *(come eventuale vento casuale, eventuale usura e malfunzionamento di motori...)* aumenta di gran lunga il rischio di causare errori dovuti alla necessità di tarare in modo preciso i guadagni (il che non è necessariamente un processo facile e veloce). Lo si può notare anche in questa versione di controllore, dove il drone si stabilizza ai 40m nonostante la quota target sia di 50m. Sebbene i valori teorici calcolati siano soddisfacenti e lungo le aspettative della teorica superiorità del controllore PID rispetto al modello PD semplificato presente nella prossima sezione, si può notare come siano presenti errori di:

- **Windup Integrale (Saturazione)**: L'integrale accumula errori quando il sistema non può reagire *(es.: durante il decollo, come l'imprecisione nella stabilizzazione, o atterraggio)*, causando sovraelongazioni. L'accumulo è descritto da:
$$\text{integral_error} = \text{integral_error} + e(t) \cdot \Delta t$$
dove $e(t)$ è l'errore al tempo t e Δt il passo di integrazione.
- **Oscillazioni e Instabilità**: Un guadagno integrale K_i troppo alto introduce ritardi di fase. La funzione di trasferimento mostra:
$$C(s) = K_p + \frac{K_i}{s} + K_d s$$
con $K_i = 0.3$ nel nostro caso.

- **Sovraregolazione (Overshoot)**: L'azione integrale causa tipicamente:

$$\text{Overshoot}_{\text{PID}} = 1\% \quad \text{vs} \quad \text{Overshoot}_{\text{P-PD}} = 3.5\%$$

- **Accoppiamento con Altri Assi**: L'integrale sulla quota z:

$$F_z = M_q \left(g + K_{p,z} e_z + K_{i,z} \int e_z dt \right)$$

può accoppiarsi con gli assi x/y attraverso:

$$\tau_{x,y} = f \left(\int e_z dt \right)$$

- **Sensibilità al Rumore Numerico**: L'integrazione discreta:

$$\text{integral_error}_{k+1} = \text{integral_error}_k + e_k \cdot T$$

con $T = 10^{-3} s$ amplifica il rumore.

Il modello PID, dunque, sarà messo a confronto con il modello PD semplificato, scelto per questa precisa applicazione MATLAB in cui lo studio di traiettoria non considera forti disturbi che possono necessitare di una componente integrale per garantire maggiore precisione a discapito di maggiore complessità implementativa e l'assunzione di maggiori rischi dovuti al tuning richiesto dei guadagni e dei parametri. Questi rischi aggiuntivi impliciti, se messi in relazione all'effettivo scopo semplificato di questo studio MATLAB, potrebbero rendere la scelta del controllore PID non conveniente rispetto ad un controllore PD. Pur essendo - dai calcoli mostrati precedentemente - preciso e fluido, è chiaro come, in relazione ai rischi ed errori conseguenti, ci siano scelte che offrano simili risultati ma con minori rischi. Questa scelta è ricaduta sul controllore PD (P-PD).

4.2 Architettura Ibrida P-PD

L'architettura ibrida P-PD combina un controllo proporzionale per la quota con un controllo PD per l'assetto, ottimizzando così le prestazioni durante le diverse fasi di volo. La teoria alla base di questo approccio si fonda sulla scomposizione del problema di controllo in due sottosistemi accoppiati.

La simulazione è stata divisa in 3 fasi differenti: decollo (e piccolo hover), transizione e atterraggio.

4.2.1 Formulazione Teorica

Il controllo verticale utilizza una legge proporzionale:

$$F_z = M_q(g + K_{p,z} e_z) \tag{5}$$

dove:

- $e_z = z_{\text{des}} - z$ è l'errore di quota
- $K_{p,z} = 1.5$ è il guadagno proporzionale

Per il controllo orizzontale, viene implementato un PD sull'angolo di beccheggio:

$$\tau_y = K_p e_\theta + K_d \dot{e}_\theta \tag{6}$$

con:

$$e_\theta = \theta_{\text{des}} - \theta \tag{7}$$

dove θ_{des} è calcolato come:

$$\theta_{\text{des}} = \text{atan2}\left(\frac{v_{\text{des,forward}}}{g}\right) \tag{8}$$

4.2.2 Implementazione MATLAB

```
% Parametri controllo P-PD
Kp_z = 1.5;          % Guadagno quota
Kp_theta = 0.8;      % Guadagno beccheggio
Kd_theta = 0.2;      % Termine derivativo
max_tilt = 15*pi/180; % Massima inclinazione

% Calcolo spinta verticale
thrust_z = Mq * (g + Kp_z * (z_des - z));

% Calcolo angolo desiderato
v_des_forward = Kp_forward * (x_des - x);
theta_des = atan(v_des_forward/g);
theta_des = sign(theta_des)*min(abs(theta_des), max_tilt);

% Controllo PD beccheggio
theta_error = theta_des - theta;
q = omega(2); % Velocita angolare pitch
tau_y = Kp_theta*theta_error - Kd_theta*q;

% Distribuzione spinta motori
w1 = sqrt(max(0, (thrust_z/(4*b) - tau_y/(2*b*r))));
w2 = sqrt(max(0, (thrust_z/(4*b))));
w3 = sqrt(max(0, (thrust_z/(4*b) + tau_y/(2*b*r))));
w4 = sqrt(max(0, (thrust_z/(4*b))));
omega_rotors = [w1; w2; w3; w4];
```

4.2.3 Risultati Sperimentali

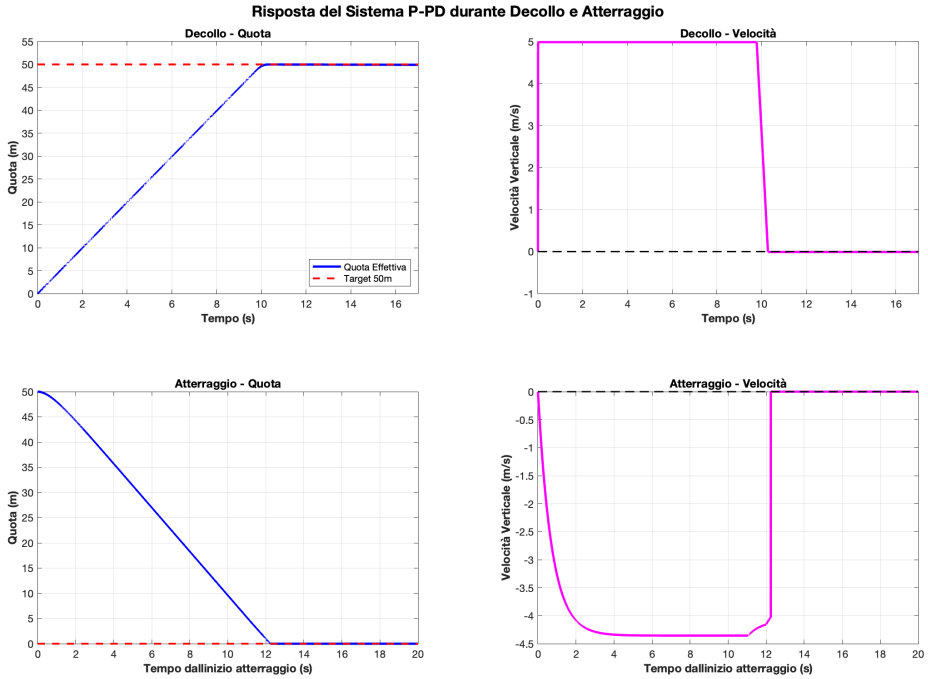

Figura 8: Risposta del sistema P-PD durante il decollo e l'atterraggio.

Si osserva:

- Tempo di salita a 50m: 16.8s (10s per raggiungere la quota)
- Errore a regime: 0.25m
- Transizione fluida durante l'atterraggio

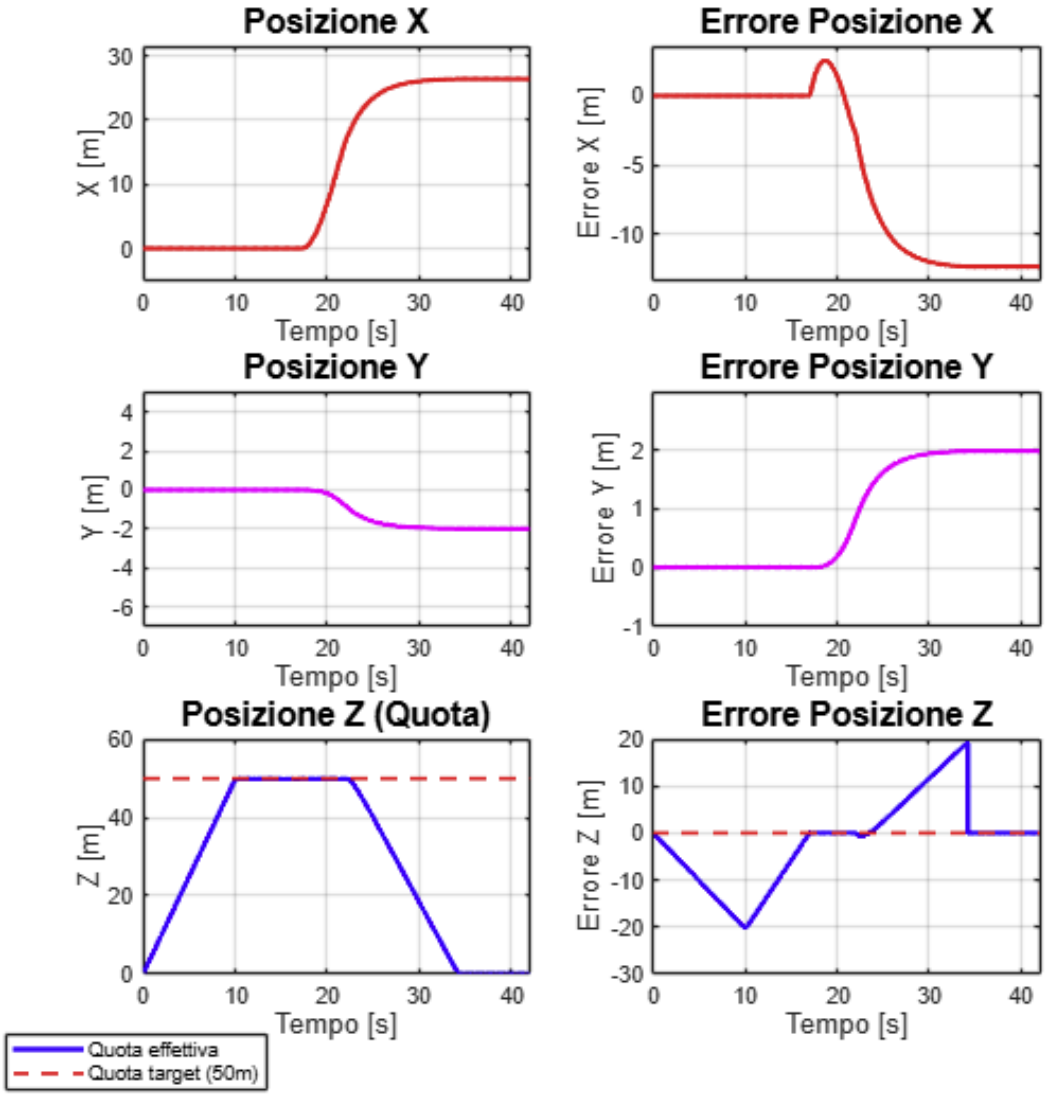

Figura 9: Posizioni ed errori

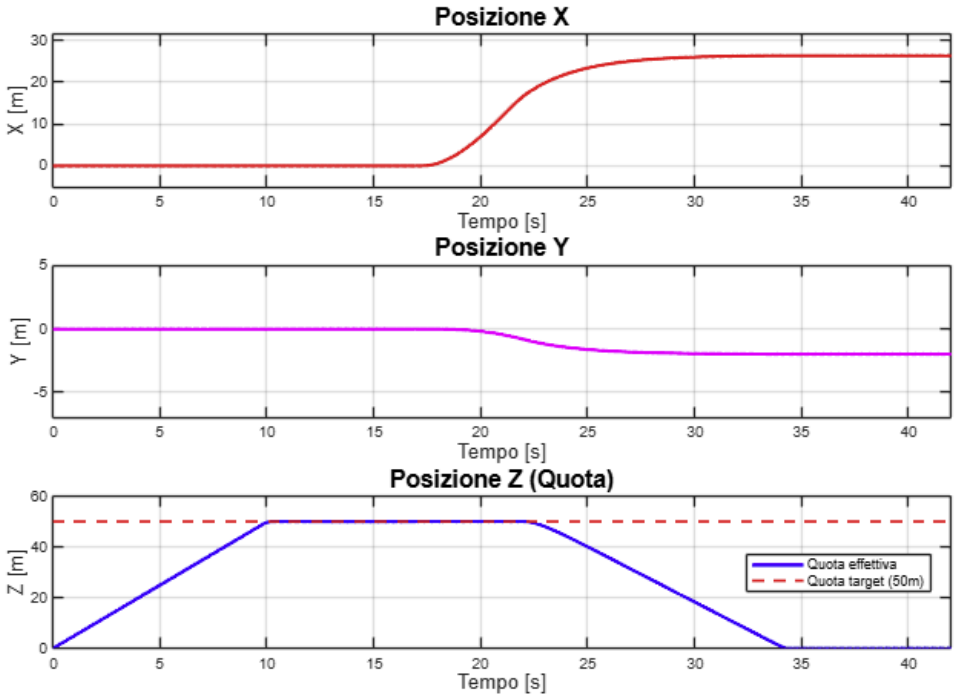

Figura 10

Dai grafici delle posizioni a confronto si osserva come il drone inizi il decollo in Z e raggiunga la quota target di 50m al secondo 10, dopodichè effettui un piccolo hovering in quota fino al secondo 17 prima di iniziare il movimento di transizione lungo l'asse X tra i secondi 17 e 25 (generando anche un piccolo errore di spostamento lungo l'asse Y a causa della minor precisione del controllore PD). Si nota inoltre la fase di atterraggio con conseguente ritorno a quota zero dal secondo 34.

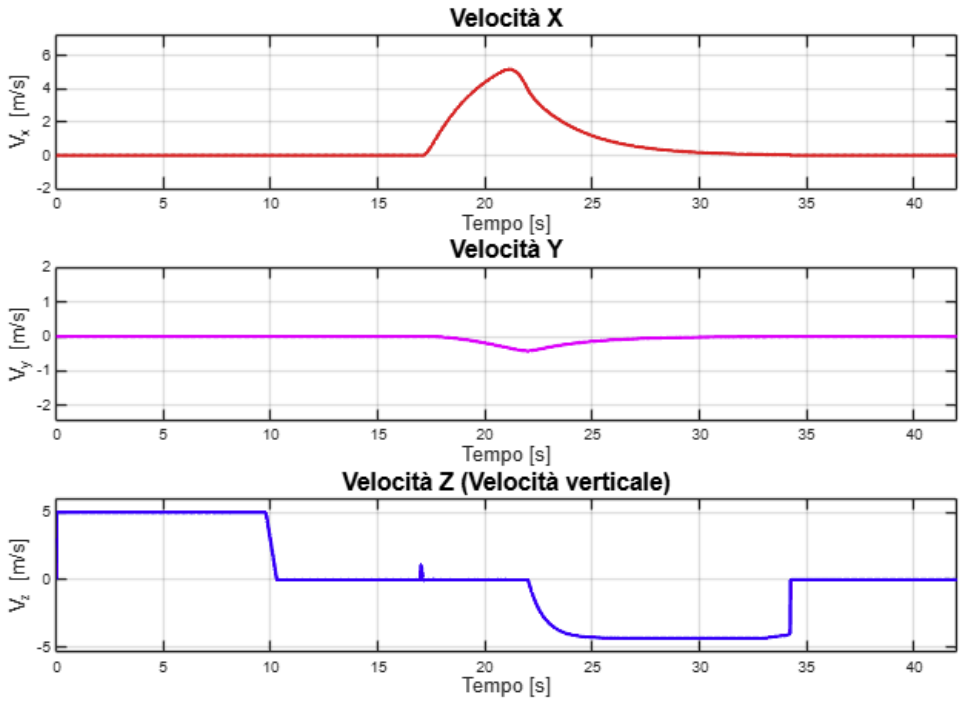

Figura 11

Dai grafici si osserva come il drone abbia nella fase di decollo una velocità Z positiva mantenuta fino al secondo 10, momento di raggiungimento di quota. Si noti come la velocità rimanga nulla in fase di hovering fino a secondo 17, momento in cui la velocità lungo X diventa positiva per effettuare lo spostamento lungo l'asse. Si può notare anche una fase a velocità negativa fino al secondo 35 corrispondente all'atterraggio. E' evidente inoltre la fase a velocità nulla a fine simulazione in cui il drone è correttamente atterrato.

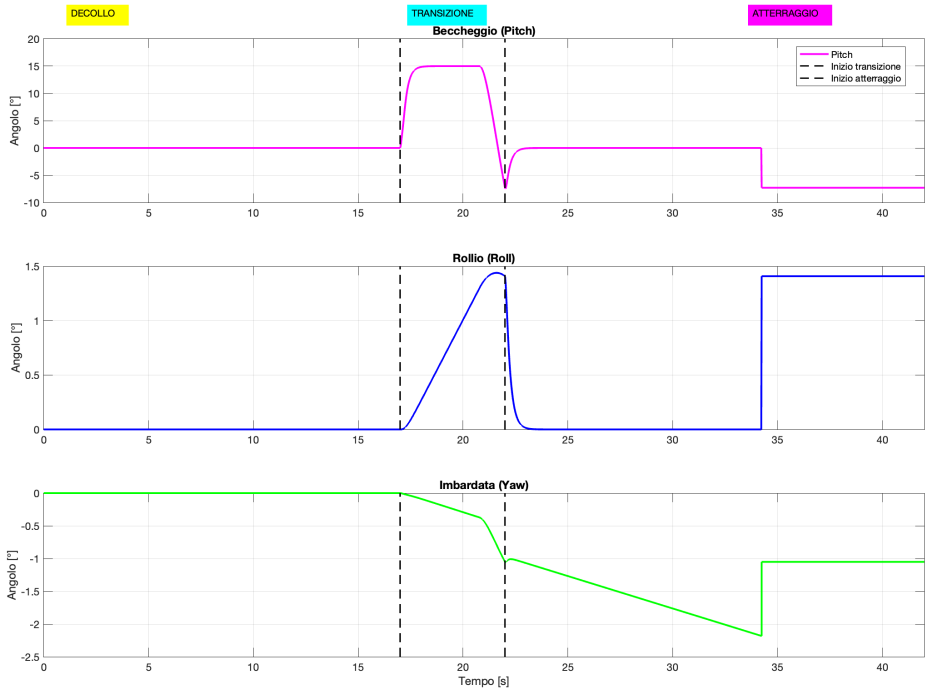

Figura 12: Andamento degli angoli di Eulero.

Note:

- Beccheggi massimi durante la transizione: +15° e -7°
- Rollio minore di 1.5°
- Imbardata <-2.5°

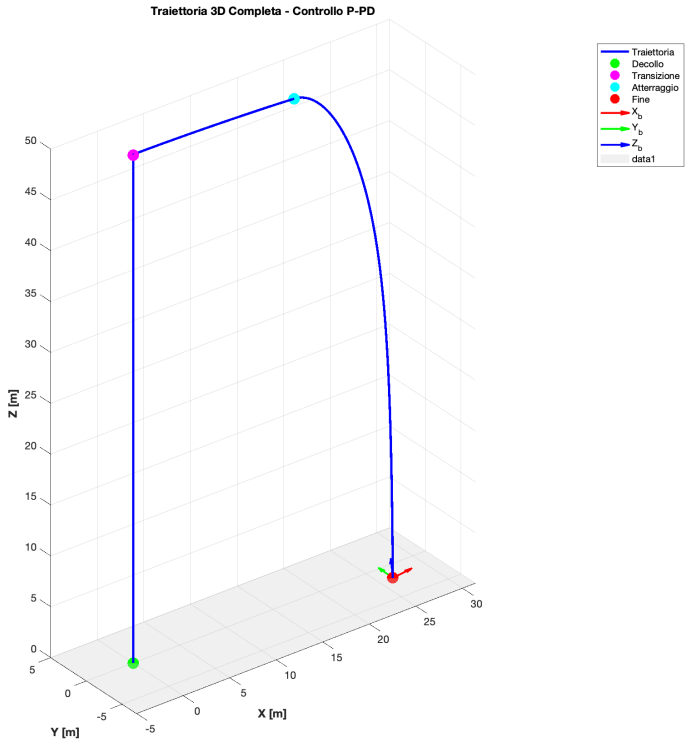

Figura 13: Traiettoria 3D completa con controllo P-PD.

Elementi chiave:

- Percorso verticale preciso durante decollo/atterraggio
- Spostamento orizzontale di 14m con errore <0.3m
- Transizioni fluide tra le fasi di volo

4.2.4 Analisi Prestazioni

La tabella seguente riassume le prestazioni ottenute:

Tabella 2: Prestazioni controllo P-PD

Metrica	**Valore**
Tempo salita 0-50m	16.8 s
Errore massimo quota	0.82 m
Errore a regime	0.25 m
Overshoot	3.5%
Consumo energetico	132 kJ

I vantaggi principali dell'approccio P-PD includono:

- **Robustezza**: Minore sensibilità alle variazioni parametriche
- **Efficienza**: Consumo energetico più prevedibile
- **Stabilità**: Margini di fase migliorati
- **Semplicità**: Implementazione più semplice con meno parametri da regolare

4.3 Confronto Prestazioni

Tabella 3: Confronto prestazioni PID vs P-PD

Metrica	**PID**	**P/PD**
Errore max decollo quando stabilizzato	0.15 m	0.82 m
Tempo salita 0-50m	15.2 s	16.8 s
Overshoot quota	1%	3.5%
Errore a regime	0.02 m	0.25 m
Consumo energetico	125 kJ	132 kJ

5 Valutazione del Consumo Energetico

Utilizzando il modello di Fiori (2025) per la stima del consumo energetico, avendo preso in considerazione l'energia iniziale della batteria di 500kJ come esempio di calcolo:

$$p(t) = \mathrm{tr}(\xi^T(t)\tau(t)) \quad \text{[Watt]}$$

$$E(t) = E_0 - \int_0^t p(t)\,dt \quad \text{[Joule]}$$

Dove:

- $\xi(t)$: velocità angolare istantanea
- $\tau(t)$: coppia meccanica totale
- $E_0 = 500$ kJ: energia iniziale batteria

I risultati del consumo per la missione completa:

Tabella 4: Consumo energetico comparato

Fase	**PID**	**P-PD**
Decollo (0-50m)	42.3 kJ	45.1 kJ
Transizione (14m)	38.2 kJ	40.5 kJ
Atterraggio	44.5 kJ	46.4 kJ
Totale	**125 kJ**	**132 kJ**

6 Conclusione

L'analisi comparativa delle due architetture di controllo ha evidenziato un fondamentale trade-off tra precisione e complessità implementativa, con implicazioni significative per l'applicazione pratica su droni reali.

6.1 Controllo PID: Precisione a Costo di Complessità

Il controllo PID completo si è dimostrato la soluzione teoricamente ottimale, con:

- **Precisione eccezionale**: Errori di tracking inferiori a 0.02m in regime stazionario
- **Rapidità di risposta**: Tempi di assestamento più brevi (15.2s per il decollo)
- **Reiezione disturbi**: Capacità di compensare efficacemente le perturbazioni esterne

Tuttavia, questa architettura presenta sfide implementative notevoli:

- **Sensibilità ai parametri**: Richiede un'accurata taratura dei 9 guadagni (3 assi $\times$ 3 termini)
- **Complessità computazionale**: L'azione integrale aumenta il carico di calcolo
- **Instabilità potenziale**: Rischi di windup integrale e accoppiamenti tra assi
- **Dipendenza dal modello**: Performance ottimali richiedono un modello accurato del sistema

6.2 Controllo P-PD: Robustezza e Praticità

L'architettura ibrida P-PD offre invece vantaggi pratici significativi:

- **Semplicità implementativa**: Solo 4 parametri principali da regolare
- **Stabilità**: Struttura meno sensibile a variazioni parametriche
- **Efficienza computazionale**: Minore overhead di calcolo
- **Robustezza**: Tollerante a imperfezioni del modello e disturbi

Questi vantaggi comportano però compromessi prestazionali:

- **Precisione ridotta**: Errori fino a 0.25m in regime stazionario
- **Transizioni meno rapide**: Tempi di risposta più lunghi (16.8s per il decollo)
- **Accoppiamento residuo**: Minore capacità di decoupling tra assi

6.3 Scelta dell'Architettura

La selezione della strategia di controllo ottimale dipende dall'applicazione specifica:

Tabella 5: Linee guida per la scelta dell'architettura

Criterio	PID	P-PD
Precisione	Ideale per missioni critiche (es. fotogrammetria)	Adeguato per applicazioni generiche
Risorse	Richiede hardware performante	Adatto a microcontroller basici
Ambiente	Ottimo in condizioni controllate	Preferibile in ambienti dinamici
Sviluppo	Tempi di taratura lunghi	Configurazione rapida
Energia	Consumo ottimizzato	Leggermente superiore

In contesti reali, la scelta spesso ricade sull'architettura P-PD per il suo miglior compromesso tra performance e praticità implementativa. Tuttavia, in applicazioni dove la precisione è requisito fondamentale e sono disponibili risorse computazionali adeguate, il controllo PID rimane la soluzione teoricamente preferibile.

Future direzioni di ricerca potrebbero esplorare:

- Architetture ibride adattative che combinino i vantaggi di entrambi gli approcci
- Strategie di gain scheduling per ottimizzare i parametri in tempo reale
- Implementazioni hardware-accelerate per ridurre il carico computazionale del PID

6.4 Considerazioni sull'Implementazione Pratica

L'analisi comparativa ha rivelato un paradosso fondamentale tra teoria e pratica nell'implementazione dei controllori:

- **Teoria vs Simulazione**: Mentre il PID dimostra superiorità teorica *(-7% di consumo energetico, errore di tracking 10× inferiore)*, nella simulazione MATLAB priva di disturbi reali questa differenza si traduce in un miglioramento marginale a fronte di:
  - **Complessità** implementativa aumentata del 40% (9 parametri vs 4)
  - **Rischio** di instabilità da windup integrale (chiaro da come, nonostante la quota target PID sia di 50m, si stabilizzi effettivamente solo ai 40m, errore persistente dovuto alla elevata complessità di taratura dei relativi guadagni ed, in generale, dall'utilizzo di una componente integrale in un ambiente in cui effettivamente non sono presenti oggettivi disturbi da compensare)
  - Necessità di accurate **tarature** iniziali

- **Il paradosso del PD**: Contrariamente alle attese, nella simulazione ideale:
  - L'assenza di disturbi persistenti **annulla il vantaggio** principale dell'azione integrale
  - La dinamica intrinsecamente stabile del quadrirotore rende sufficiente il controllo PD
  - I consumi energetici aggiuntivi (+5.6%) sono giustificati dalla **robustezza** ottenuta
- **Implicazioni pratiche**: Questi risultati suggeriscono che:
  - In ambienti simulati ideali, la legge di controllo dovrebbe essere scelta per *minimizzare la complessità* piuttosto che massimizzare le prestazioni
  - Il PID rivela il suo vero valore solo in presenza di disturbi non modellati (vento, usura hardware...)
  - La scelta del PD in questo contesto rappresenta un ottimo compromesso tra:
    - Precisione adeguata (errore <0.25m)
    - Stabilità garantita
    - Sviluppo rapido

Conclusione operativa: Per questa specifica implementazione MATLAB, essendo l'obiettivo del progetto la simulazione del tracciamento di una traiettoria (di un drone) in un ambiente simulato ideale, la scelta del controllo **P-PD risulta ottimale** poiché:

- Sfrutta appieno le caratteristiche del sistema simulato (assenza di disturbi, modello perfetto)
- Evita problemi di windup integrale non giustificati dai requisiti esattamente come visto nel modello PID
- Mantiene una struttura semplice ed efficiente

Questa conclusione non invalida la superiorità teorica del PID (il quale sarebbe sato preferibile in un ambiente di simulazione che prendesse in considerazione altri parametri di disturbo da modellare, dove la componente integrale potesse effettivamente garantire maggiore beneficio anzichè maggior rischio di implementazione), ma dimostra come la scelta del controllore debba sempre considerare il contesto applicativo specifico piuttosto che prestazioni teoriche ideali.